

Arquitectura de Computadores - Resumen de clase

Clase del 16 de abril - MASM/x64 + introduccion a IEEE-754

Material base: video abril-16.mp4 y archivo 4-16.pdf. Este resumen esta orientado a repasar rapido y poder resolver consultas/practicass de ensamblador.

1. Idea central de la clase

La clase cierra el ejercicio de la suma de cubos y luego abre el tema de representacion de numeros con parte fraccionaria. En la primera parte se termina y depura el programa MASM que verifica la identidad:

$$1^3 + 2^3 + \dots + n^3 = (1 + 2 + \dots + n)^2 = (n(n+1)/2)^2$$

- Se completa formulaCuadradoSuma para calcular $(n(n+1)/2)^2$.
- Se revisa como imprimir en consola usando GetStdHandle y WriteFile de kernel32.dll.
- Se depura paso a paso en Visual Studio observando registros, memoria, RSP/RBP y el resultado en hexadecimal.
- Se introduce IEEE-754 como la forma estandar de representar reales/fraccionarios en binario.

2. Que habia que entender del programa MASM

El programa compara dos maneras de calcular el mismo valor para un limite n:

Parte	Responsabilidad	Dato importante
main	Coordina las llamadas: obtiene n, calcula sumaCubos, calcula formulaCuadradoSuma y compara.	R8 se usa para la suma de cubos; R9 para la formula.
getSumLimit	Devuelve el limite n y muestra un mensaje en consola.	Usa pila para el valor de retorno y Windows API para imprimir.
sumCubes	Calcula $1^3 + 2^3 + \dots + n^3$ con un ciclo.	Usa variables locales i y s dentro del stack frame.
formulaCuadradoSuma	Calcula $(n(n+1)/2)^2$.	El resultado se devuelve en R9w/R9d segun la version.
anunciarIgualdad / anunciarDesigualdad	Deberian anunciar el resultado al usuario.	En el esqueleto pueden quedar vacias o implementarse con WriteFile.

3. Flujo del main

El main funciona como una traduccion casi directa de un algoritmo de alto nivel:

```
; 1) Reservar espacio para el valor retornado por getSumLimit
```

```
sub RSP, 8
```

```
call getSumLimit
```

```
pop RCX      ; RCX recibe n
```

```
mov n, CX
```

```
; 2) Calcular la suma de cubos
```

```
xor R8, R8
```

```
call sumCubes ; retorna en R8
```

```
mov SumaCubos, R8d
```

```
; 3) Calcular la formula del cuadrado de la suma
xor R9, R9
call formulaCuadradoSuma ; retorna en R9
```

```
; 4) Comparar ambos resultados
mov R8d, SumaCubos
cmp R8, R9
jne elseComparar
call anunciarIgualdad
jmp endIfResultadosIguales
elseComparar:
call anunciarDesigualdad
endIfResultadosIguales:
call ExitProcess
```

Detalle clave: se guarda SumaCubos en memoria antes de llamar formulaCuadradoSuma porque esa funcion puede usar R8 como registro auxiliar y destruir el valor anterior. Luego se recarga R8d antes del cmp.

4. getSumLimit: pila + llamadas a Windows API

En esta clase se vuelve importante la convencion de llamadas de Windows x64. Para llamar funciones externas como GetStdHandle y WriteFile:

- Los primeros cuatro argumentos se pasan en RCX, RDX, R8 y R9.
- El llamador debe reservar 32 bytes de shadow space antes del call.
- Si hay un quinto argumento, se coloca en la pila, normalmente en [RSP+32] despues de reservar el shadow space.
- GetStdHandle devuelve el handle en RAX.
- WriteFile recibe handle, direccion del mensaje, longitud, direccion donde escribir bytes escritos y NULL/overlapped.

```
sub RSP, 32          ; shadow space para llamada Windows x64
mov RCX, -11         ; STD_OUTPUT_HANDLE: salida de consola
call GetStdHandle    ; RAX = handle de consola
mov RBX, RAX
```

```
mov RCX, RBX         ; 1er argumento: handle
lea RDX, mensaje1     ; 2do argumento: direccion del texto
mov R8, msg1Len       ; 3er argumento: longitud
lea R9, bytesEscritos ; 4to argumento: variable de salida
mov qword PTR [RSP+32], 0 ; 5to argumento: NULL
call WriteFile
add RSP, 32
```

```
mov dword PTR [RSP+8], limit ; valor de retorno para el caller
ret
```

Ojo: -11 corresponde normalmente a `STD_OUTPUT_HANDLE`. Si en algun comentario aparece como `STDIN`, lo mas probable es que sea un error de comentario.

5. sumCubes: variables locales dentro del stack frame

sumCubes crea dos variables locales en la pila: i y s. La idea en pseudocodigo seria:

```
i = 0
s = 0
while i != n:
    i = i + 1
    s = s + i*i*i
return s
```

En ensamblador, el procedimiento guarda una referencia al inicio del stack frame en RBP, reserva 8 bytes y usa esos bytes como dos variables de 32 bits:

```
mov RBP, RSP
sub RSP, 8
mov dword PTR [RSP], 0 ; i = 0
mov dword PTR [RSP+4], 0 ; s = 0
```

cicloSumaCubos:

```
xor EAX, EAX
xor EDX, EDX
inc dword PTR [RSP] ; i++
mov EBX, [RSP]
mov EAX, EBX
mul EBX ; EDX:EAX = i*i
mul EBX ; EDX:EAX = i*i*i
add [RSP+4], EAX ; s += i^3
cmp [RSP], ECX ; i == n?
jne cicloSumaCubos
```

```
xor R8, R8
mov R8d, [RSP+4] ; return en R8d
mov RSP, RBP ; eliminar variables locales
ret
```

- RSP apunta a la cima de la pila.
- RBP se usa como ancla para restaurar RSP al final.
- mul usa EAX implícitamente: multiplica EAX por el operando y coloca el resultado en EDX:EAX.
- El resultado final se mueve a R8d porque el main espera la suma de cubos en R8.

6. formulaCuadradoSuma: calcular $(n(n+1)/2)^2$

La formula que se implementa es:

$$(1 + 2 + \dots + n)^2 = (n(n+1)/2)^2$$

Version vista en clase, siguiendo el enfoque con registros de 16 bits:

```
formulaCuadradoSuma PROC
    mov RBP, RSP
    xor RAX, RAX
    mov AX, CX      ; AX = n
    inc AX          ; AX = n + 1
    mul CX          ; DX:AX = n(n+1)
    mov R10w, 2
    xor RDX, RDX
    idiv R10w       ; AX = n(n+1)/2
    mov R8w, AX     ; auxiliar = suma simple
    mul R8w         ; DX:AX = suma simple^2
    mov R9w, AX     ; return en R9w
    ret
formulaCuadradoSuma ENDP
```

Version mas robusta para estudiar, usando 32 bits y evitando que R8 destruya el resultado anterior:

```
formulaCuadradoSuma PROC
    movzx EAX, CX   ; EAX = n
    mov EDX, EAX    ; EDX = n
    inc EDX         ; EDX = n + 1
    imul EAX, EDX   ; EAX = n(n+1)
    shr EAX, 1      ; EAX = n(n+1)/2
    imul EAX, EAX   ; EAX = (n(n+1)/2)^2
    xor R9, R9
    mov R9d, EAX    ; return en R9d
    ret
formulaCuadradoSuma ENDP
```

- Para $n = 5$, la suma de cubos es $1 + 8 + 27 + 64 + 125 = 225 = 0xE1$.
- La formula da $(5*6/2)^2 = 15^2 = 225 = 0xE1$.
- En la depuracion, R8 y R9 terminan con el mismo valor, por eso no se toma el jne y se ejecuta anunciarIgualdad.

- Si se usan registros de 16 bits, el programa sirve para valores pequenos. Para valores mayores hay riesgo de truncamiento/overflow.

7. Errores y detalles tipicos que conviene revisar

Detalle	Por que importa
kernel32.dll	Debe escribirse kernel32.dll, no kernel132.dll.
CuadradoSuma dd 3	Debe haber espacio entre dd y el valor. CuadradoSuma dd3 es error.
STD_OUTPUT_HANDLE	Para imprimir en consola se usa -11. No confundirlo con entrada estandar.
RSP balanceado	Todo sub RSP, X debe compensarse con add RSP, X o restaurando RSP desde RBP.
Registros destruidos	Si una funcion auxiliar usa R8, y el main ocupaba conservar R8, hay que guardarlo en memoria o pila.
Tamanos de registro	R9w son 16 bits; R9d son 32 bits; R9 son 64 bits. Comparar R8 con R9 puede fallar si las partes altas no estan limpias.

8. Segunda parte de la clase: IEEE-754 y numeros fraccionarios

Despues de terminar el ejercicio de MASM, la clase cambia a la representacion de numeros con parte fraccionaria. La idea principal: los enteros y los reales no se representan igual. Para reales se usa IEEE-754, basado en notacion cientifica normalizada.

- En decimal, un numero puede escribirse como mantisa x $10^{\text{exponente}}$, por ejemplo 2.99792458×10^8 .
- En binario, la forma normalizada se parece a $1.\text{xxxx} \times 2^e$.
- El bit de signo indica positivo o negativo.
- El primer 1 de la mantisa normalizada suele ser implicito, porque en numeros normalizados siempre es 1.
- La parte fraccionaria/mantisa guarda los bits despues del punto binario.
- El exponente indica cuanto se mueve el punto binario.

La diapositiva mostro un ejemplo del estilo:

$$-1.0001010010 \times 2^{-47}$$

Esto se interpreta como signo negativo, mantisa 1.0001010010 y exponente -47. En IEEE-754 el valor no se guarda como texto, sino como campos binarios: signo, exponente y fraccion/mantisa.

9. Conversion de fraccion decimal a binario

La clase termino con una demostracion en Excel usando un numero como pi. La tecnica para convertir la parte fraccionaria es multiplicar repetidamente por 2:

1. Separar la parte entera y la parte fraccionaria. Ejemplo: 3.14159265 -> entero 3 y fraccion 0.14159265.
2. Multiplicar la fraccion por 2.
3. El entero del resultado es el siguiente bit binario.
4. Quitar esa parte entera y repetir con la nueva fraccion.
5. La secuencia de bits obtenida forma la parte fraccionaria en binario.

Paso	Fraccion x 2	Bit tomado	Nueva fraccion
1	$0.14159265 \times 2 = 0.28318530$	0	0.28318530
2	$0.28318530 \times 2 = 0.56637060$	0	0.56637060

Paso	Fraccion x 2	Bit tomado	Nueva fraccion
3	$0.56637060 \times 2 = 1.13274120$	1	0.13274120
4	$0.13274120 \times 2 = 0.26548240$	0	0.26548240

Idea importante: muchas fracciones decimales no tienen una representacion binaria finita. Por eso los numeros de punto flotante son aproximaciones y pueden aparecer pequenos errores de redondeo.

10. Checklist para consultas o trabajo

- Antes de programar, definir donde va cada parametro y donde se devuelve el resultado.
- Si llamo una funcion, revisar que RSP quede igual al salir que antes de reservar espacio local.
- Si uso una API de Windows, reservar 32 bytes de shadow space y pasar argumentos en RCX, RDX, R8, R9.
- Si una funcion usa registros auxiliares, no asumir que conservan valores anteriores.
- Para depurar, revisar registros, memoria y banderas despues de cmp.
- Para IEEE-754, separar signo, mantisa y exponente; para fracciones, practicar multiplicaciones sucesivas por 2.

11. Mini guia de instrucciones vistas

Instruccion	Uso	Ejemplo mental
mov	Mover/copiar datos	mov AX, CX -> AX recibe CX
lea	Cargar direccion efectiva	lea RDX, mensaje -> RDX apunta al mensaje
xor reg, reg	Limpiar un registro	xor R8, R8 -> R8 = 0
sub/add RSP, n	Reservar/liberar espacio en pila	sub RSP, 8 crea espacio local
call	Invocar procedimiento	call sumCubes
ret	Regresar al caller	ret usa la direccion de retorno de la pila
cmp	Comparar y actualizar banderas	cmp R8, R9
jne	Saltar si no son iguales	jne elseComparar
mul	Multiplicacion sin signo usando A implicitamente	mul EBX -> EDX:EAX = EAX*EBX
idiv	Division con signo usando DX:AX / divisor	idiv R10w -> cociente en AX
shr	Desplazar a la derecha; divide por 2 si es positivo	shr EAX, 1